2026학년도 2학기 인문데이터 수량분석

제5강 탐색적 자료 분석 (3)

박수민

서울대학교 인문대학 연계전공 인문데이터과학

2026년 9월 16일 수요일

오늘의 목표

1. 두 표가 어떤 열로 이어지는지(키) 찾아낼 수 있다.
2. `left_join()`으로 흩어진 정보를 하나의 표로 모을 수 있다.
3. `anti_join()`으로 짝이 맞지 않는 행을 찾아낼 수 있다.
4. `pivot_longer()`와 `pivot_wider()`로 표의 모양을 바꿀 수 있다.

사전 준비

1. Posit Cloud의 `qah-2026` 프로젝트를 연다.
2. `hdr/data/`에 `acs_cbsa_to_state.csv`, `acs_state.csv`, `food_prices.csv`가 있는지 확인한다.
3. 콘솔에서 `library(tidyverse)`와 `library(ggrepel)`을 실행해 둔다.

1 **여러 표를 잇기**
왜 표가 나뉘어 있는가
키와 관계
`left_join`

2 조인의 종류

3 표의 모양 바꾸기

4 어떤 모양으로 저장할 것인가

5 마무리

한 표에 다 넣으면 안 되는가

도서관 목록을 한 장의 표로 만든다면

『무정』, 『흙』, 『사랑』… 이광수의 책마다 생몰년과 출생지를 다시 적어야 한다.

반복이 부르는 것

- ✗ 생몰년이 틀렸다면 **모든 행**을 고쳐야 한다.
- ✗ 한 곳만 고치면 같은 사람의 생몰년이 두 가지가 된다.
- ✗ 책이 한 권도 없는 작가는 아예 적을 자리가 없다.

그래서 나누어 둔다

- 작품 표에는 **작품**에 관한 것만, 작가 표에는 **작가**에 관한 것만 적는다.
- 둘을 잇는 열(작가 번호)을 하나 둔다.

→ 분석할 때 그 열을 열쇠 삼아 **다시 붙인다**.

❗ 인문 데이터베이스는 거의 전부 이렇게 생겼다. 오늘 배울 것은 그것을 **되돌려 붙이는** 방법이다.

한 표 동사와 두 표 동사

4강에서 배운 것: 한 표 동사

`filter`, `select`, `mutate`... 모두 **표 하나**를 받아 표 하나를 돌려주었다.

오늘 배울 것: 두 표 동사

표 둘을 받아 표 하나를 돌려준다. 이름이 모두 `_join`으로 끝난다.

달라지는 것은 하나뿐

- 첫 번째 인자가 표라는 점, 원본을 바꾸지 않는다는 점, 파이프로 이어 붙일 수 있다는 점은 그대로다.
- → **두 번째 인자로 또 다른 표**가 들어온다. 그리고 `by =`로 어느 열을 열쇠로 삼을지 알려 준다.

💡 오늘의 후반부에서 배울 `pivot` 함수는 다시 한 표 동사다. 표를 합치는 것이 아니라 **모양**을 바꾼다.

오늘의 세 표

```
library(tidyverse)

cbsa   <- read_csv(file.path("hdr", "data", "acs_cbsa.csv"))
lookup <- read_csv(file.path("hdr", "data", "acs_cbsa_to_state.csv"))
state  <- read_csv(file.path("hdr", "data", "acs_state.csv"))
```

오늘은 934곳 전부

4강에서 `filter(cbsa, quad != "0")`로 916곳만 남겨 두었다면, 위 코드로 **다시 읽어** 934곳으로 되돌린다.

왜 되돌리는가

오늘은 주(state)별로 묶어 볼 참이다. 하와이와 알래스카를 빼 두면 그 두 주의 계산이 어긋난다.

⚠ 자료를 걸러 둔 채로 잊어버리는 것은 흔한 실수다. 중간에 무엇을 뺐는지 코드에 남겨 두어야 이런 일을 막을 수 있다.

연결 표: lookup

```
# A tibble: 996 x 3
  geoid state    prop
  <dbl> <chr>   <dbl>
1 35620 NY    0.648
2 35620 NJ    0.349
3 35620 PA    0.00303
4 31080 CA    1
5 16980 IL    0.907
6 16980 IN    0.0757
7 16980 WI    0.0177
# i 989 more rows
```

한 도시가 여러 주에 걸친다

대도시권은 경제권이지 행정구역이 아니다. 뉴욕 대도시권(35620)은 뉴욕주·뉴저지주·펜실베이니아주에 걸쳐 있다.

prop

그 대도시권 인구 가운데 해당 주에 사는 사람의 **비율**이다. 한 geoid의 prop을 모두 더하면 1이 된다.

- 934곳이 996행이 된 까닭은, 여러 주에 걸친 도시가 주마다 한 행씩 차지하기 때문이다.

주 정보 표: state

```
# A tibble: 50 x 3
   state       abb       pop
   <chr>       <chr>   <dbl>
 1 Alabama     AL       5.00
 2 Alaska      AK       0.736
 3 Arizona     AZ       7.08
 4 Arkansas    AR       3.01
 5 California  CA      39.5
 6 Colorado    CO       5.72
 # i 44 more rows
```

50개 주

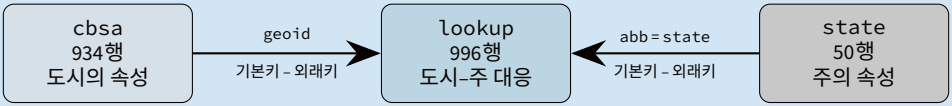
주의 이름(state), 두 글자 약칭(abb), 총인구(pop, 백만 명)다.

lookup과 이어지는 곳

lookup의 state 열("NY")과 state의 abb 열("NY")에 같은 값이 들어 있다.

⚠ 두 표 모두 state라는 이름의 열을 갖고 있지만 **담긴 내용이 다르다**. 뒤에서 이 때문에 한 번 걸려 넘어진다.

기본키와 외래키



기본키(primary key)

그 표의 한 행을 **유일하게** 가리키는 열이다. cbsa에서는 geoid, state에서는 abb다.

외래키(foreign key)

남의 표의 기본키가 내 표에 얹혀 있는 것이다. lookup의 geoid와 state가 둘 다 외래키다.

❗ cbsa의 name은 기본키가 아니다. 같은 이름을 가진 대도시권이 여럿 있기 때문이다.

작은 표로 연습하기

```
cbsa_sml <- cbsa |>
  select(name, geoid, pop, density)
```

```
# A tibble: 934 x 4
  name         geoid   pop density
  <chr>        <dbl> <dbl>   <dbl>
1 New York     35620 20.0    1051.
2 Los Angeles 31080 13.2    1041.
3 Chicago      16980  9.61    509.
# i 931 more rows
```

- 열이 13개나 되면 붙인 결과가 화면을 넘어간다. 그래서 네 열만 남겼다.
- 조인의 결과를 눈으로 확인할 때는 이렇게 `select`로 줄여 두면 편하다.

두 표 붙이기: left_join()

```
lookup |>
  left_join(cbsa_sml, by = "geoid")
```

```
# A tibble: 996 x 6
  geoid state    prop name          pop density
  <dbl> <chr>   <dbl> <chr>       <dbl>   <dbl>
1 35620 NY    0.648   New York    20.0    1051.
2 35620 NJ    0.349   New York    20.0    1051.
3 35620 PA    0.00303 New York    20.0    1051.
4 31080 CA    1       Los Angeles 13.2    1041.
5 16980 IL    0.907   Chicago      9.61    509.
6 16980 IN    0.0757  Chicago      9.61    509.
# i 990 more rows
```

- 왼쪽 표(lookup)의 996행이 그대로 남고, 오른쪽 표의 열 셋이 뒤에 붙었다.
- 뉴욕의 인구 20.0이 **세 번** 나타난다. 같은 geoid를 가진 행이 셋이므로 그만큼 복사된 것이다.

키 이름이 서로 다를 때

```
lookup |>
  left_join(state, by = c("state" = "abb"))
```

```
# A tibble: 996 x 5
  geoid state    prop state.y        pop
  <dbl> <chr>   <dbl> <chr>        <dbl>
1 35620 NY    0.648   New York     20.1
2 35620 NJ    0.349   New Jersey    9.23
3 35620 PA    0.00303 Pennsylvania 13.0
4 31080 CA    1       California   39.5
5 16980 IL    0.907   Illinois     12.8
# i 991 more rows
```

읽는 법

`c("왼쪽 이름" = "오른쪽 이름")`. 왼쪽 표의 state와 오른쪽 표의 abb를 맞추라는 뜻이다.

state.y는 무엇인가

두 표에 state라는 열이 둘 다 있었다. 한 표에 같은 이름의 열이 둘일 수는 없으므로 R이 이름을 바꾼 것이다.

이름 충돌 정리하기: suffix

```
lookup |>
  left_join(
    state, by = c("state" = "abb"), suffix = c("", "_name")
  )
```

```
# A tibble: 996 x 5
  geoid state    prop state_name     pop
  <dbl> <chr>   <dbl> <chr>        <dbl>
1 35620 NY    0.648   New York     20.1
2 35620 NJ    0.349   New Jersey    9.23
3 35620 PA    0.00303 Pennsylvania 13.0
# i 993 more rows
```

- suffix는 두 글자다. **왼쪽 표에 붙일 꼬리표**와 **오른쪽 표에 붙일 꼬리표**. ""는 아무것도 붙이지 않는다는 뜻이다.
- 이름이 겹치지 않는 열에는 꼬리표가 붙지 않는다. prop은 그대로다.
- 기본값은 `c(".x", ".y")`다. 뜻이 통하지 않으니 직접 지어 주는 편이 낫다.

1 여러 표를 잇기

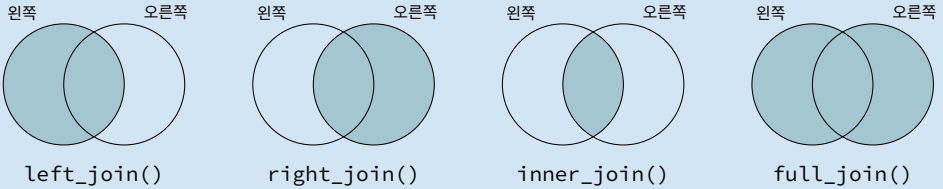
2 **조인의 종류**
네 가지 변형
필터링 조인
세 표 잇기

3 표의 모양 바꾸기

4 어떤 모양으로 저장할 것인가

5 마무리

짝이 없는 행을 어떻게 할 것인가



- 색이 칠해진 부분이 **결과에 남는 행**이다. 가운데는 양쪽에 짝이 있는 행이다.
- 짝이 없는 채로 남은 행의 빈칸은 NA(결측)로 채워진다.

! 붙는 **열**은 네 경우 모두 같다. 다른 것은 남는 **행**뿐이다.

어느 것을 쓸 것인가

함수	남는 행	쓰는 때
`left_join()`	왼쪽 전부	왼쪽 표에 정보를 **덧붙일** 때
`inner_join()`	양쪽에 짝이 있는 것만	짝 없는 행을 **버려야** 할 때
`right_join()`	오른쪽 전부	거의 쓰지 않는다
`full_join()`	양쪽 전부	두 목록을 **합칠** 때

지켜야 할 원칙 하나

오른쪽 표에서 키가 기본키가 되도록 한다. 즉 오른쪽 표에는 같은 키를 가진 행이 하나뿐이어야 한다.

- 이 원칙만 지키면 실제로 필요한 것은 `left_join`과 `inner_join` 둘뿐이다.
- ⚠ 원칙을 어기면 행이 **불어난다**. 996행을 붙였는데 3,000행이 나왔다면 이것을 의심한다.

붙이지 않고 거르기만 하는 조인

`semi_join()`

오른쪽 표에 **짝이 있는** 행만 남긴다. 열은 하나도 늘지 않는다.

`anti_join()`

오른쪽 표에 **짝이 없는** 행만 남긴다. 역시 열은 그대로다.

`filter`의 확장이다

`filter`는 조건을 직접 써서 거른다. 필터링 조인은 **다른 표에 있는지 여부**로 거른다.

언제 쓰는가

- “연구 대상 목록에 있는 작품만”
- “아직 정리하지 못한 항목만”

💡 이름이 `join`으로 끝나지만 하는 일은 `filter`에 가깝다. 그래서 **필터링 조인**이라 부른다.

짝이 맞는지 검사하기

```
lookup |>
  anti_join(state, by = c("state" = "abb"))
```

```
# A tibble: 1 x 3
  geoid state  prop
  <dbl> <chr> <dbl>
1 47900 DC    0.105
```

무엇이 걸려 나왔나

워싱턴 D.C.다. `lookup`은 이것을 주 코드처럼 다루지만, `state`에는 없다. 50개 주에 들지 않기 때문이다.

조인 전에 반드시

`left_join` 결과에서 `NA`를 발견하고 당황하기 전에, `anti_join`으로 **먼저** 확인한다.

- 결과가 0행이면 모든 행에 짝이 있다는 뜻이다. 그때 비로소 안심하고 붙인다.
- 인명·지명 표기가 조금씩 다른 자료를 이을 때 이 검사가 특히 요긴하다. “金正喜”와 “김정희”는 컴퓨터에게 다른 글자다.

질문: 어느 주가 가장 도시적인가

```
lookup |>
  inner_join(cbsa, by = "geoid") |>
  group_by(state) |>
  summarize(pop = sum(pop * prop)) |>
  inner_join(
    state, by = c("state" = "abb"), suffix = c("", "_full")
  ) |>
  mutate(percentage = pop / pop_full * 100) |>
  arrange(desc(percentage)) |>
  select(state_full, percentage)
```

- 두 표씩 두 번 이었다. 세 표를 한 번에 잇는 함수는 없다.
- 긴 파이프가 두려우면 **한 줄씩 지우면서** 실행해 본다. 어느 단계에서 표가 어떻게 생겼는지 보이면 전체가 읽힌다.

한 줄씩 읽어 보기

여섯 단계

1. `inner_join` — 도시의 인구를 붙인다.
2. `group_by` — 주별로 묶는다.
3. `summarize` — 인구 × 비율을 더한다.
4. `inner_join` — 주의 총인구를 붙인다.
5. `mutate` — 백분율을 계산한다.
6. `arrange`, `select` — 정렬해 두 열만.

```
# A tibble: 50 x 2
   state_full    percentage
   <chr>              <dbl>
 1 New Jersey         101.
 2 Delaware           101.
 3 Hawaii             100.0
 4 Connecticut        100.0
 5 Rhode Island        99.9
 6 Massachusetts       99.7
 7 California          99.3
 # i 43 more rows
```

`sum(pop * prop)`이 핵심이다. 뉴욕 2,000만 명 가운데 뉴저지 몫 34.9%만 뉴저지에 더한다.

결과 읽기

작고 도시적인 주들

뉴저지, 델라웨어, 하와이, 코네티컷. 주민 거의 전부가 어느 대도시권 안에 산다. 반대쪽 끝은 몬태나, 메인, 버몬트, 알래스카다.

100%가 넘는 이유

두 자료의 조사 시점과 추계 방법이 조금 다르다. 반올림도 겹친다.

그래서 어떻게 하는가

101%를 100%로 고쳐 적지 않는다. **그런 오차가 있는 자료**라고 밝히고 쓴다.

⚠ 조인은 서로 다른 곳에서 온 자료를 잇는 일이다. 이을 수 있다는 것이 곧 **같은 것을 재고 있다**는 뜻은 아니다.

➔ 출처가 다른 두 표를 이었다면, 무엇과 무엇을 이었는지 반드시 적어 둔다.

인문학 자료를 관계로 보기

한국고전종합DB, 한국사데이터베이스, 국립중앙도서관 서지

모두 여러 표로 나뉘어 있다. 문헌 표, 인물 표, 지명 표, 연호 표…

연구 질문이 곧 조인이다

"조선왕조실록 기사를 **왕대별**로 세어 보자"는 기사 표에 왕대 표를 잇는 일이다.

인문학의 어려움

이름이 여럿이고(자·호·시호), 지명이 바뀌고, 연호가 겹친다. 기계적인 키가 잘 만들어지지 않는다.

➔ 그래서 인문 데이터에서는 **전거(典據) 관리**, 곧 하나의 대상에 하나의 식별자를 붙이는 일이 그 자체로 연구가 된다.

💡 국제적으로는 VIAF, 국내에서는 한국역사정보통합시스템의 인물 식별자가 그런 시도다.

1 여러 표를 잇기

2 조인의 종류

3 **표의 모양 바꾸기**
넓은 표와 긴 표
pivot_longer
pivot_wider

4 어떤 모양으로 저장할 것인가

5 마무리

같은 자료, 두 가지 모양

넓은 표 (2행 3열)

	tea	rice	corn
1950	68.9	62.7	135.
1975	54.1	74.2	113.

pivot_longer →

← pivot_wider

긴 표 (6행 3열)

year	food	price
1950	tea	68.9
1950	rice	62.7
1950	corn	135.
1975	tea	54.1
1975	rice	74.2
1975	corn	113.

- 담긴 값은 똑같다. **잃는 것도 얻는 것도 없다.** 놓이는 자리만 바뀐다.
- 넓은 표에서는 열 이름(`tea`)이 **정보**다. 긴 표에서는 그것이 값이 된다.

오늘의 자료: food_prices

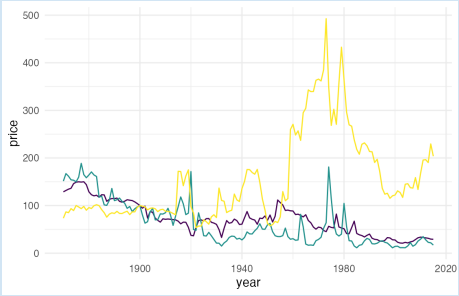
```
1 food_prices <- read_csv(file.path("hdr", "data", "food_prices.csv"))
```

```
# A tibble: 146 x 14
   year   tea sugar peanuts coffee cocoa wheat   rye  rice
  <dbl> <dbl> <dbl>   <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>
1  1870  129.  151.    203.   88.1  78.8  88.1  103.  83.5
2  1871  132.  167.    222.  109.   66.7 118.   105.  84.5
3  1872  134.  162.    189.  140.   71.6 122.   102.  92.9
# i 143 more rows
# i 5 more variables: corn <dbl>, barley <dbl>, pork <dbl>,
#   beef <dbl>, lamb <dbl>
```

- 1870년부터 2015년까지 13가지 식료품의 **실질 가격 지수**다. 1900년의 가격을 100으로 삼았다.
- 한 행이 한 해, 한 열이 한 품목이다. 전형적인 **넓은 표**다.

넓은 표로 그림을 그려 보면

```
food_prices |>
  ggplot(aes(x = year)) +
    geom_line(aes(y = tea),
              color = "#440154") +
    geom_line(aes(y = sugar),
              color = "#21918c") +
    geom_line(aes(y = beef),
              color = "#fde725")
```



✗ 세 품목에 세 층. 열세 품목이면 열세 층이고, 색도 열세 개를 손으로 고른다.

✗ **범례가 없다.** 어느 선이 무엇인지 그림만 보고는 알 수 없다.

➔ 3강에서 배운 대로다. 색은 `aes()` **안**에서 열에 이어야 범례가 생긴다. 그러려면 품목이 **하나의 열**이어야 한다.

길게 만들기: pivot_longer()

```
food_prices |>
  pivot_longer(-c(year))
```

```
# A tibble: 1,898 x 3
   year name    value
  <dbl> <chr>   <dbl>
1  1870 tea     129.
2  1870 sugar   151.
3  1870 peanuts 203.
4  1870 coffee   88.1
5  1870 cocoa    78.8
# i 1,893 more rows
```

무엇을 적어 주는가

값으로 바뀔 열이 아니라, 값으로 바뀌지 **않을** 열을 적는다. `-c(year)`의 빼기 기호가 그 뜻이다.

146행이 1,898행으로

$146 \times 13 = 1{,}898$. 열세 개의 열이 열세 개의 행으로 풀렸다.

⚠ 새 열의 이름이 `name`과 `value`다. 기본값이라 뜻이 없다.

이름을 붙여 주기

```
food_prices |>
  pivot_longer(-c(year), names_to = "food", values_to = "price")
```

```
# A tibble: 1,898 x 3
   year food    price
  <dbl> <chr>   <dbl>
1  1870 tea     129.
2  1870 sugar   151.
3  1870 peanuts 203.
4  1870 coffee   88.1
# i 1,894 more rows
```

names_to

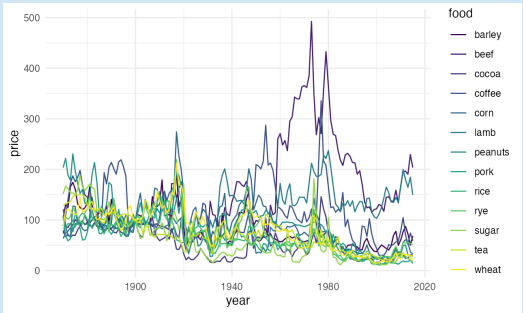
열 이름들이 옮겨 갈 새 열의 이름이다.
여기서는 품목이다.

values_to

칸 안의 값들이 옮겨 갈 새 열의 이름이다.
여기서는 가격이다.

한 층으로 열세 개의 선을

```
food_prices |>
  pivot_longer(-year,
    names_to = "food",
    values_to = "price") |>
  ggplot() +
    geom_line(
      aes(x = year, y = price,
          color = food)) +
    scale_color_viridis_d()
```



✓ 층은 하나, 색은 자동, 범례도 자동이다. 고친 것은 그림이 아니라 **자료의 모양**이다.

💡 `scale_color_viridis_d()`의 d는 이산형(discrete)이다. 3강에서 배운 그대로다.

반대 방향: 두 해만 골라내기

```
food_prices_long <- food_prices |>
  pivot_longer(-year, names_to = "food", values_to = "price") |>
  filter(year %in% c(1950, 1975))
```

```
# A tibble: 26 x 3
   year food    price
  <dbl> <chr>   <dbl>
1  1950 tea      68.9
2  1950 sugar    59.5
3  1950 peanuts 119.
4  1950 coffee  210.
# i 22 more rows
```

- 13품목 × 2개 연도 = 26행이다.

→ 1950년과 1975년의 가격을 **두 축에 놓고** 비교하고 싶다. 그러려면 두 해가 각각 **열**이어야 한다.

넓게 만들기: pivot_wider()

```
food_prices_long |>
  pivot_wider(
    names_from = year,
    values_from = price
  )
```

```
# A tibble: 13 x 3
   food     `1950` `1975`
   <chr>     <dbl>  <dbl>
 1 tea        68.9   54.1
 2 sugar      59.5  114.
 3 peanuts   119.    95.6
 4 coffee    210.   105.
 5 cocoa      64.6   54.1
# ... (8행 생략)
```

names_from

새 열 이름을 꺼내 올 열이다. year의 값 1950과 1975가 두 개의 열 이름이 된다.

values_from

칸을 채울 값을 꺼내 올 열이다.

pivot_longer와 짝

names_to/values_to가 “어디로”, names_from/values_from이 “어디서”다.

숫자로 시작하는 열 이름

```
food_prices_long |>
  pivot_wider(
    names_from = year,
    values_from = price,
    names_prefix = "year_"
  )
```

```
# A tibble: 13 x 3
  food     year_1950 year_1975
  <chr>        <dbl>     <dbl>
1 tea           68.9      54.1
2 sugar         59.5     114.
3 peanuts      119.       95.6
4 coffee       210.      105.
# ... (9행 생략)
```

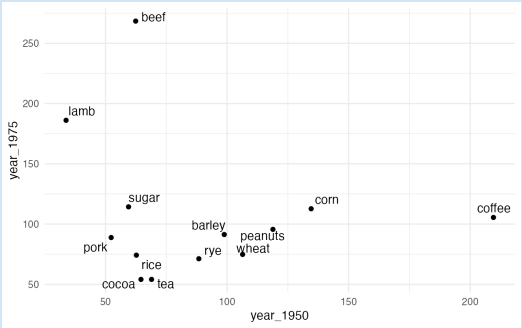
앞 슬라이드의 백틱은 무엇이었나

`1950`처럼 감싸인 것은 **쓸 수 없는 이름**이라는 표시다. R에서 이름은 숫자로 시작할 수 없다.

- names_prefix가 앞에 year_를 붙여 정상적인 이름으로 만들어 준다.
- 백틱을 매번 붙여 가며 쓸 수도 있지만, 오타가 나기 쉽다. 이름을 고치는 편이 낫다.

25년 사이에 무엇이 비싸졌나

```
food_prices_long |>
  pivot_wider(
    names_from = year,
    values_from = price,
    names_prefix = "year_"
  ) |>
  ggplot(aes(year_1950,
             year_1975)) +
    geom_point() +
    geom_text_repel(
      aes(label = food))
```



➔ 소고기·양고기는 크게 올랐고 커피는 크게 내렸다.

- 두 해를 **두 축**에 놓으려면 두 해가 두 열이어야 한다.

1 여러 표를 잇기

2 조인의 종류

3 표의 모양 바꾸기

4 **어떤 모양으로 저장할 것인가**

5 마무리

긴 형태를 기본으로

원칙

자료는 되도록 긴 형태로 저장하고, 필요할 때 넓게 만든다.

긴 형태의 이점

- 열이 적으니 **열마다 무슨 뜻인지** 설명해 두기 쉽다.
- 품목이 하나 늘어도 표의 구조가 바뀌지 않는다. 행만 늘어난다.
- `ggplot2`와 `dplyr`이 모두 긴 형태를 전제로 만들어졌다.

넓은 형태가 나은 때

- 사람이 눈으로 읽는 표. 신문의 통계표는 언제나 넓다.
- 두 시점, 두 집단을 **나란히 놓고** 비교할 때.

→ 둘 다 **마지막 단계**의 일이다.

⚠ 넓게 저장해 두면 분석할 때마다 길게 폈다가 다시 넓히는 일을 반복하게 된다.

정돈된 데이터(tidy data)

세 가지 약속

1. 한 **열**이 하나의 변수를 담는다.
2. 한 **행**이 하나의 관측을 담는다.
3. 한 **칸**에 하나의 값이 들어간다.

넓은 `food_prices`는 왜 어긋나는가

`tea`, `sugar`라는 열 이름 자체가 "품목"이라는 변수의 **값**이다.

긴 표는 어떤가

`year`, `food`, `price`. 한 행이 "어느 해 어느 품목의 값"이라는 하나의 관측이다.

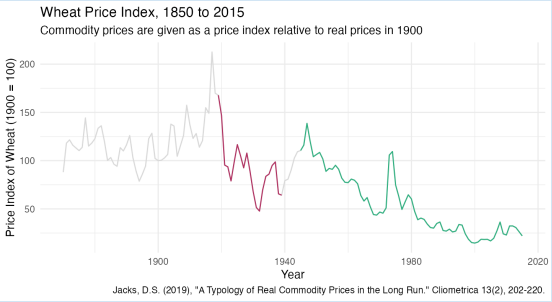
- "정돈되었다"는 것은 깨끗하다는 뜻이 아니라 **이 세 약속을 지킨다**는 뜻이다.
- `tidyverse`라는 이름이 여기서 왔다. 이 원칙에 맞는 자료를 **처음부터 만드는** 방법은 6강에서 배운다.

종합: 밀 가격 200년

```
food_prices |>
  ggplot(aes(year, wheat)) +
    geom_line(color = "grey85") +
    geom_line(color = "maroon",
      data = filter(food_prices, between(year, 1919, 1939))) +
    geom_line(color = "#30b080",
      data = filter(food_prices, between(year, 1945, 2015))) +
    labs(
      title = "Wheat Price Index, 1850 to 2015",
      subtitle = "Commodity prices are given as a price index ...",
      caption = "Jacks, D.S. (2019), ...",
      x = "Year", y = "Price Index of Wheat (1900 = 100)"
    )
```

- 4강에서 배운 `data =`로 층마다 다른 구간을 칠했다. 여기서는 **넓은 표**가 편하다. `wheat` 한 열만 쓰기 때문이다.

두 전쟁 뒤의 밀값



두 시기

- 1차 대전 뒤(자주색): 떨어졌다가 요동친다.
- 2차 대전 뒤(초록색): 꾸준히 내려간다.

💡 회색으로 전체를 깔고 관심 구간만 칠했다. 맥락을 보이면서 시선을 이끈다.

➔ 그림 안에 설명을 적고 싶다면 R로 씨름하기보다 슬라이드 도구가 빠르다.

국내 통계표를 다룰 때

KOSIS, 한국은행 경제통계, 문화체육관광부 통계

내려받은 엑셀 파일은 거의 언제나 **연도가 열**로 늘어선 넓은 표다. 사람이 읽기 위한 모양이기 때문이다.

첫 손질

`pivot_longer`로 연도를 하나의 열로 모은다. 그래야 선그래프도, 집단별 요약도 가능해진다.

자주 걸리는 것

- 연도가 `"2020년"`처럼 글자로 들어온다.
- 제목 행이 두세 줄씩 겹쳐 있다.
- 결측이 `"-"`나 `"..."`로 적혀 있다.

➔ 이런 것을 손보는 방법이 6강 “데이터 수집”의 내용이다.

1 여러 표를 잇기

2 조인의 종류

3 표의 모양 바꾸기

4 어떤 모양으로 저장할 것인가

5 **마무리**

오늘의 함수 한눈에

함수	하는 일	결과
`left_join()`	왼쪽을 다 두고 붙이기	열↑
`inner_join()`	짝이 있는 것만 붙이기	열↑, 행↓
`full_join()`	양쪽을 다 두고 붙이기	열↑, 행↑
`semi_join()`	짝이 있는 행만 남기기	행↓
`anti_join()`	짝이 없는 행만 남기기	행↓
`pivot_longer()`	열 이름을 값으로 내리기	행↑, 열↓
`pivot_wider()`	값을 열 이름으로 올리기	행↓, 열↑

💡 `join`은 `dplyr`, `pivot`은 `tidyr`의 함수다. 둘 다 `tidyverse`에 딸려 온다.

오늘 배운 것

관계와 키

- 기본키와 외래키, 그리고 관계
- by =로 키 지정하기, 이름이 다르면 `c("a" = "b")`
- 이름이 겹치면 suffix

표의 모양

- 넓은 표와 긴 표는 같은 자료다
- names_to/values_to, names_from/values_from
- names_prefix로 이름 고치기

조인의 종류

- 짝 없는 행을 어떻게 할 것인가
- anti_join으로 붙이기 전에 검사하기

정돈된 데이터

- 한 열에 한 변수, 한 행에 한 관측
- 긴 형태로 저장하고 필요할 때 넓히기

확인 질문

스스로 답해 보기

1. `lookup`에 `cbsa_sml`을 붙였더니 뉴욕의 인구가 세 번 나타났다. 왜인가?
2. `by = c("state" = "abb")`에서 `"state"`는 어느 표의 열인가?
3. 붙이기 전에 `anti_join`을 먼저 실행해 보는 이유는 무엇인가?
4. `pivot_longer`를 한 뒤 표의 행 수와 열 수는 각각 어떻게 되는가?
5. 넓은 표를 그대로 두고 열세 개의 선을 그리면 무엇이 없는가?

💡 답이 막히는 항목이 있으면 eTL Q&A 게시판에 질문을 올린다 (참여 점수 1점).

다음 시간에 배울 것

- 제6강 데이터 수집 [교재 5장]
  - 분석 가능한 자료를 처음부터 설계하는 법
  - 결측값과 불확실한 정보를 일관되게 기록하기

다음 시간까지 할 일

1. 관심 있는 공개 데이터를 하나 내려받아, 넓은 표인지 긴 표인지 판단해 보기
2. 그 표를 `pivot_longer`로 길게 만들어 선그래프를 하나 그려 보기

[숙제04] 데이터 재구조화 (3점)

- 파일명: `hw04-2026-00000.Rmd` (`2026-00000`은 자신의 학번)
- 제출 기한: **2026-09-28 월요일 09:00** eTL
- 오늘 배운 내용으로 충분히 풀 수 있다. 자세한 안내는 eTL 공지 참조.
- Knit이 오류 없이 되는지 반드시 확인하고 제출할 것!

참고 자료

- `dplyr` 조인 치트시트 — `https://rstudio.github.io/cheatsheets/data-transformation.pdf`
- `tidyr` 치트시트 — `https://rstudio.github.io/cheatsheets/tidyr.pdf`